

HDOC Day-39 Activity

C Programming - Variadic Functions and Command-Line Arguments

Question 1: Weighted Average Using a Variadic Function

Write a C program containing the variadic function `double weightedAverage(int count, ...)`. The function accepts count and pairs of values and weights, validates the weights, handles zero total weight, and returns the weighted average.

Algorithm

1. Start the program and define `weightedAverage(int count, ...)`.
2. Initialize the variable-argument list using `va_start()`.
3. For each pair, read a value and its weight using `va_arg()`.
4. If any weight is negative, display an error, clean up the argument list, and return an error value.
5. Accumulate value x weight and the total weight.
6. If the total weight is zero, display an error and return an error value.
7. Otherwise, return `weighted sum / total weight`.
8. Call the function from `main()` and display the result.

C Program

```
#include <stdio.h>
#include <stdarg.h>

double weightedAverage(int count, ...)
{
    va_list args;
    double weightedSum = 0.0, totalWeight = 0.0;
    double value, weight;
    int i;

    if (count <= 0) {
        printf("Error: count must be positive.\n");
        return -1.0;
    }

    va_start(args, count);

    for (i = 0; i < count; i++) {
        value = va_arg(args, double);
        weight = va_arg(args, double);

        if (weight < 0.0) {
            printf("Error: weights must be non-negative.\n");
            va_end(args);
            return -1.0;
        }

        weightedSum += value * weight;
        totalWeight += weight;
    }

    if (totalWeight == 0.0) {
        printf("Error: total weight is zero.\n");
        return -1.0;
    }

    return weightedSum / totalWeight;
}
```

```

    }

    weightedSum += value * weight;
    totalWeight += weight;
}

va_end(args);

if (totalWeight == 0.0) {
    printf("Error: total weight cannot be zero.\n");
    return -1.0;
}

return weightedSum / totalWeight;
}

int main(void)
{
    double result;

    result = weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5);

    if (result >= 0.0)
        printf("Weighted Average: %.2f\n", result);

    return 0;
}

```

Sample Output

Weighted Average: 84.00

Calculation: $(80 \times 0.3 + 75 \times 0.2 + 90 \times 0.5) / (0.3 + 0.2 + 0.5) = 84.00$

Test Case Table

Test Case	Input / Function Call	Expected Output / Behaviour	Result
1	weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5)	Weighted Average: 84.00	Pass
2	weightedAverage(2, 50.0, 0.5, 100.0, 0.5)	Weighted Average: 75.00	Pass
3	weightedAverage(2, 50.0, -0.2, 80.0, 0.5)	Error: weights must be non-negative.	Pass
4	weightedAverage(2, 50.0, 0.0, 80.0, 0.0)	Error: total weight cannot be zero.	Pass

Question 2: Process Integer Command-Line Arguments

Write a C program that accepts any number of command-line arguments representing integers. Non-integer arguments are ignored. The valid integers are sorted in ascending order, then the minimum, maximum, average, median, and second-largest distinct value are displayed.

Algorithm

9. Read command-line arguments from argv[].
10. Validate each argument as a complete integer. Ignore and report invalid arguments.
11. Store all valid integers in an array.
12. Sort the valid integers in ascending order.
13. Display the sorted integers.
14. Calculate minimum, maximum, average, and median.
15. Scan backward through the sorted array to find the second-largest distinct value.
16. Display all calculated results.

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <limits.h>

int isInteger(const char *text, int *value)
{
    char *end;
    long number;

    errno = 0;
    number = strtol(text, &end, 10);

    if (text[0] == '\0' || *end != '\0' || errno == ERANGE ||
        number < INT_MIN || number > INT_MAX)
        return 0;

    *value = (int)number;
    return 1;
}

void sortAscending(int a[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

```

    }
}
}

double calculateAverage(const int a[], int n)
{
    long long sum = 0;
    int i;
    for (i = 0; i < n; i++)
        sum += a[i];
    return (double)sum / n;
}

double calculateMedian(const int a[], int n)
{
    if (n % 2 == 1)
        return a[n / 2];
    return ((double)a[n / 2 - 1] + a[n / 2]) / 2.0;
}

int secondLargestDistinct(const int a[], int n, int *answer)
{
    int i;
    for (i = n - 2; i >= 0; i--) {
        if (a[i] != a[n - 1]) {
            *answer = a[i];
            return 1;
        }
    }
    return 0;
}

int main(int argc, char *argv[])
{
    int *numbers, count = 0, value, secondLargest, i;

    if (argc < 2) {
        printf("No arguments provided.\n");
        return 0;
    }

    numbers = malloc((argc - 1) * sizeof(int));
    if (numbers == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

```

```

for (i = 1; i < argc; i++) {
    if (isInteger(argv[i], &value))
        numbers[count++] = value;
    else
        printf("Invalid argument ignored: %s\n", argv[i]);
}

if (count == 0) {
    printf("No valid integers provided.\n");
    free(numbers);
    return 0;
}

sortAscending(numbers, count);

printf("Valid integers in ascending order:");
for (i = 0; i < count; i++)
    printf(" %d", numbers[i]);
printf("\n");

printf("Minimum: %d\n", numbers[0]);
printf("Maximum: %d\n", numbers[count - 1]);
printf("Average: %.2f\n", calculateAverage(numbers, count));
printf("Median: %.2f\n", calculateMedian(numbers, count));

if (secondLargestDistinct(numbers, count, &secondLargest))
    printf("Second-largest distinct value: %d\n", secondLargest);
else
    printf("Second-largest distinct value: Not available\n");

free(numbers);
return 0;
}

```

Compilation and Execution

```

gcc program.c -o program
./program 12 5 abc 8 12 20 3

```

Sample Output

```

Invalid argument ignored: abc
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00

```

Second-largest distinct value: 12

Test Case Table

Test Case	Input / Function Call	Expected Output / Behaviour	Result
1	<code>./program 12 5 abc 8 12 20 3</code>	Ignores abc; sorted: 3 5 8 12 12 20; min 3; max 20; avg 10.00; median 10.00; second-largest 12	Pass
2	<code>./program 4 1 9 2</code>	Sorted: 1 2 4 9; min 1; max 9; avg 4.00; median 3.00; second-largest 4	Pass
3	<code>./program 7 7 7</code>	Sorted: 7 7 7; min/max 7; avg/median 7.00; second-largest not available	Pass
4	<code>./program abc xyz</code>	Both ignored; No valid integers provided.	Pass
5	<code>./program -5 10 0 10</code>	Sorted: -5 0 10 10; min -5; max 10; avg 3.75; median 5.00; second-largest 0	Pass

Result

Both C programs were implemented using appropriate user-defined functions. The first program correctly uses `stdarg.h` to calculate a weighted average with input validation. The second program validates command-line integers, sorts them, and calculates the required statistical values, including the second-largest distinct value.